

recording_manager.py

전체 코드

```
# recording_manager.py

import os
import cv2
from datetime import datetime

class RecordingManager:
    def __init__(
        self,
        base_dir,
        fps=30,
        frame_size=None,
        segment_seconds=60 # 몇초단위로 영상 저장할건지
    ):
        self.base_dir = base_dir # 녹화본 저장 경로
        self.fps = fps
        self.frame_size = frame_size
        self.segment_seconds = segment_seconds

        self.writer = None # 실제로 mp4 파일에 프레임을 쓰는 객체

        self.recording_dir = os.path.join( # 기본 경로 아래에 원본 녹화본 폴더를 만
들겠다는 뜻
            self.base_dir,
            "원본 녹화본"
        )

        os.makedirs(self.recording_dir, exist_ok=True)

        self.start_time = None
        self.start_time_str = None # 녹화 시작시간 파일 이름 형식에 맞게 변환
        self.temp_save_path = None # 임시 파일 저장 경로

    def start_recording(self, frame_size): # 새로운 mp4파일 저장하는 함수
        self.frame_size = frame_size # 현재 프레임의 크기 저장

        self.start_time = datetime.now()
        self.start_time_str = self.start_time.strftime("%Y-%m-%d_%H-%M-%S") #
녹화 시작시간 파일 이름 형식에 맞게 변환
```

```

# 처음 저장시 임시파일 이름으로 저장.
temp_filename = f"recording_{self.start_time_str}.mp4"

# 임시 저장 경로 설정
# 예시) D:/AI_CCTV/원본 녹화본/recording_2026-05-20_13-30-10.mp4
self.temp_save_path = os.path.join(
    self.recording_dir,
    temp_filename
)

# 저장 코덱 설정
fourcc = cv2.VideoWriter_fourcc(*"mp4v")

# VideoWriter객체 생성
self.writer = cv2.VideoWriter(
    self.temp_save_path, # 저장 경로
    fourcc, # 코덱
    self.fps, # FPS
    self.frame_size # 프레임 크기
)

# VideoWriter가 제대로 열리지 않았으면 실패 처리. 성공하면 함수 True반환
if not self.writer.isOpened():
    print("원본 영상 저장 Writer 생성 실패")
    self.writer = None
    return False

print(f"원본 영상 저장 시작: {self.temp_save_path}")

return True

# 프레임 하나를 영상파일에 전송하는 함수
# 메인 루프에서 매 프레임마다 호출. 프레임 없으면 저장할 게 없으니 종료
def write_frame(self, frame):
    if frame is None:
        return

    # 프레임 크기 계산
    height, width = frame.shape[:2]
    current_frame_size = (width, height)

    # 아직 저장 중인 mp4 파일이 없으면 현재 프레임 크기로 새 mp4 파일을 만들기.
    # (초기에는 writer가 없으니)
    if self.writer is None:
        self.start_recording(current_frame_size)

```

```

# start_recording()을 호출했는데도 실패하면 프레임 저장 포기하고 종료
if self.writer is None:
    return

# 녹화 경과 시간 계산
elapsed_seconds = (datetime.now() - self.start_time).total_seconds()

# 지정한 segment시간 경과하면 파일 이름을 시작시간~종료시간.mp4로 변경하고 새로운 임시파일에 저장 시작
if elapsed_seconds >= self.segment_seconds:
    self.stop_recording()
    self.start_recording(current_frame_size)

# 현재 프레임을 mp4 파일에 저장
# 이 코드가 실제로 한 프레임씩 영상을 쌓는 부분
if self.writer is not None:
    self.writer.write(frame)

# 현재 저장 중인 영상 파일을 종료하는 함수
def stop_recording(self):
    # 저장 중인 파일이 없으면 할 일이 없으니까 바로 종료
    if self.writer is None:
        return

    # release를 호출해서 writer닫기. 안하면 파일 깨지거나 정상 재생 안됨.
    self.writer.release()
    self.writer = None

    # 종료시간 문자열 만들기(파일 이름용)
    end_time_str = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")

    # 임시파일 이름을 시작시간~종료시간으로 바꿔서 저장
    final_filename = f"{self.start_time_str}~{end_time_str}.mp4"

    # 최종 저장 경로 만들기
    final_save_path = os.path.join(
        self.recording_dir,
        final_filename
    )

    # 파일 이름 변경중 문제 발생시 예외처리(권한 문제, 경로 문제등등)
    try:
        os.rename(
            self.temp_save_path,
            final_save_path

```

```

    )

    print(f"원본 영상 저장 종료: {final_save_path}")

except Exception as e:
    print(f"파일 이름 변경 실패: {e}")

# 녹화 끝나면 관련 정보 초기화
self.start_time = None
self.start_time_str = None
self.temp_save_path = None

```

클래스 역할

녹화 파일 저장 폴더를 만들고 프레임을 일정 시간 간격으로 mp4파일로 저장.
 설정한 시간이 지나면 현재 파일을 닫고 새 mp4 파일로 분할 저장.

함수별 역할

start_recording()

- 입력값 : frame_size - 프레임 크기(해상도)
- 프로세스
 - 현재 프레임 크기(해상도) 저장
 - 녹화 시작시간 저장
 - 파일 이름용 문자열과 임시 mp4이름 생성
 - 저장 경로 생성, 코덱 설정
 - VideoWriter객체 생성
 - 생성 여부 True/False반환
- 출력값
 - True : 녹화 시작 성공
 - False : 녹화 시작 실패

write_frame()

- 입력값 : frame - CV영상 프레임
- 프로세스
 - frame존재 여부 확인
 - 현재 프레임 크기 계산 `height, width = frame.shape[:2]`
 - 첫 프레임 들어오면 자동 녹화 시작 - writer객체 없으면
 - 녹화 시작 실패시 종료
 - 녹화 시간 경과 계산 -> 저장 단위 시간 체크

- 단위시간 넘으면 저장 종료후 새 파일 생성
- 현재 프레임 mp4파일에 저장
- 출력값 없음

stop_recording()

- 입력값 없음
- 프로세스
 - writer존재 여부 확인 - 녹화 여부 확인
 - VideoWriter종료 - `self.writer.release()` 로 안전하게 마무리
 - writer 초기화 - `self.writer = None`
 - 종료시간 문자열 생성
 - 최종 파일 이름(시작 시간~종료시간)생성
 - 저장 경로 생성
 - 임시 파일 이름 최종 파일 이름으로 변경
 - 이름 변경시 권한 문제, 경로 문제등 예외처리
 - 다음 녹화를 위해 상태 초기화
- 출력값 없음

지피티가 짜준 핵심 정리

함수	입력값	주요 처리	출력값
<code>__init__()</code>	저장경로, FPS 등	초기 설정 저장	없음
<code>start_recording()</code>	<code>frame_size</code>	새 mp4 생성	True / False
<code>write_frame()</code>	<code>frame</code>	프레임 저장	없음
<code>stop_recording()</code>	없음	저장 종료 및 파일명 변경	없음